



Universidad de los Andes
Departamento de Ingeniería de Sistemas y Computación

Proyecto 1 - Analítica de Texto

Grupo 12
Ana María Hernández Lasso - 202220870
William Andrés Bayona Vergara - 202011494
Juan Sebastian Ardila López - 202110171

Inteligencia de Negocios

Bogotá, Colombia
2025-20

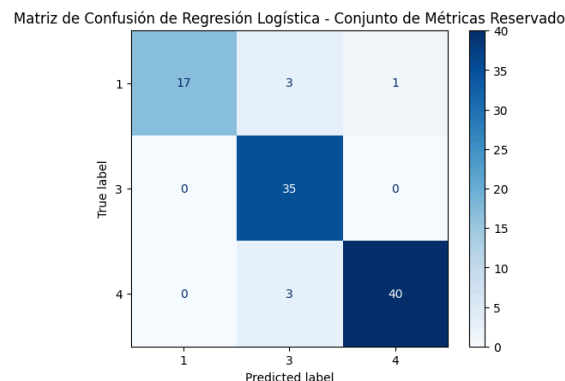
Tabla de Contenidos

- 1. Documentación de la Sección 1: Proceso de aumentación de los datos con prompting. Reentrenamiento y comparación de los modelos**
- 2. Documentación de la Sección 2: Proceso de automatización del proceso de preparación de datos, construcción y persistencia del modelo, y acceso por API**
- 3. Descripción de Tres Estrategias de Reentrenamiento**
- 4. Documentación de la Sección 3: Desarrollo de la aplicación y Justificación Descripción del Usuario de la aplicación**
- 5. Desarrollo de la Aplicación Web**
- 6. Resolución de Preguntas**
- 7. Resultados**
- 8. Pruebas de Usabilidad**
- 9. Roles y tareas realizadas por los integrantes del grupo**
- 10. Retos enfrentados y formas planteadas para resolverlo**
- 11. Declaración de uso de Inteligencia Artificial**

Proyecto 1 - Analítica de Texto

1. Documentación de la Sección 1: Proceso de aumentación de los datos con prompting. Reentrenamiento y comparación de los modelos

El primer paso para reentrenar el modelo fue realizar la prueba con los nuevos datos brindados en esta entrega y evidenciar el desempeño del modelo ante datos que no había visto. La matriz de confusión obtenida en este caso fue la siguiente:



Si bien observamos que el modelo sigue siendo funcional y permite predecir con gran eficiencia la mayoría de los datos (accuracy del 0.919), si observamos que casi todas las métricas presentan una caída que sugiere que nuestro modelo tiene espacio de mejora. Al observar la distribución de clases de los datos de la entrega 1, observamos que la clase 1 presenta la mitad de datos que la clase 4, lo cual sugiere un desbalance considerable. Ante esto, se decidió aumentar los registros de esta clase por medio de prompting. Para esto, se utilizó el API-KEY inicial brindado por el equipo académico del curso (unificado) de Open-AI. Se utilizó el modelo gpt-4o-mini y el prompt utilizado para la generación de datos fue el siguiente:

```
prompt = f"""Genera 500 opiniones, 1-2 oraciones, en español, de problemáticas locales Requisitos: - TODAS deben corresponder al Fin de la pobreza. - No salud, educación - Varía zonas (urbano/rural/paises) y actores - Neutralidad política - No contenido sensible-datos personales - Entrega SOLO JSON válido: una lista de objetos con: "textos" (string) y "labels" (int 1).Ejemplos (NO COPIAR, SOLO CONTEXTO): {ejemplos} """
```

Podemos observar que el prompt tiene un parámetro llamado *ejemplos*. Dicho parámetro es generado a través de tomar aleatoriamente 8 registros de la clase minoritaria y unificarlos en un solo string. En esta parte, era importante definir la temperatura del modelo, para esto, se probaron distintos valores y se analizaron las respuestas obtenidas. En el caso de temperaturas menores a 0.5, el modelo generaba respuestas que mantenían una estructura muy homogénea, lo cual no es deseable, pues es poco común que muchas personas sigan una estructura de texto de forma natural.

	textos	labels
2424	En zonas rurales de México, la falta de acceso...	1
2425	En las áreas urbanas de Argentina, el aumento ...	1
2426	En Colombia, la informalidad laboral es un gra...	1
2427	En Perú, las comunidades indígenas enfrentan d...	1

En este caso, se puede observar que las respuestas del modelo podrían generar sesgos en el modelo, pues no representan verdaderamente la variedad con la que se contaba en el set de datos original. Por otro lado, cuando se tienen temperaturas mayores a 1, el modelo empezaba a generar respuestas que podrían incluso romper las reglas de nuestro prompt, por ejemplo:

	textos	labels
2424	En las comunidades rurales de Colombia, la ofe...	1
2425	En México, las micronegocios enfrentan obstácu...	1
2426	La escasez de infraestructura en varias zonas ...	1
2427	En las zonas urbanas de Venezuela, la inflació...	1

El registro 2540 incluye términos como "lesbianas" y "gais", a pesar de que el prompt solicitaba contenido sensible y neutralidad política.

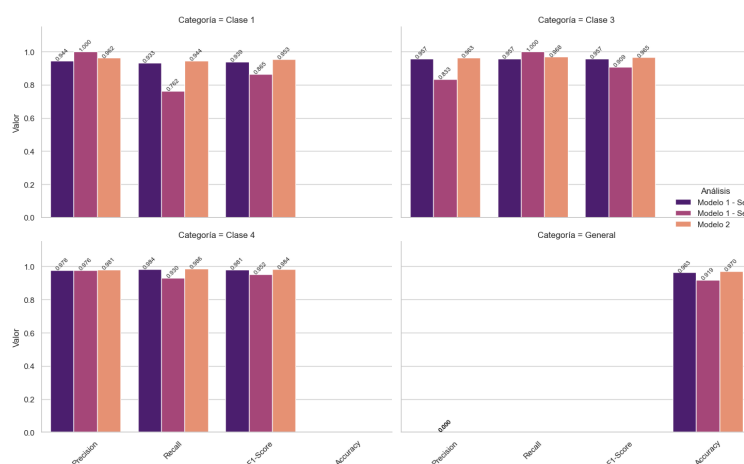
Las respuestas creativas que respetaron las reglas se obtuvieron con un umbral entre 0.7 y 0.9, lo que coincide con fuentes literarias como Holtzman y Buys (2020), quienes sugieren 0.7 para la generación de datos ("sampling").

Se solicitaron 500 opiniones a la IA, obteniendo 256 registros únicos. Se utilizaron 1136 tokens en el prompt y 10719 en la respuesta (11855 en total). Tras la adición de estos datos, la relación de clases es 1:1.36, mucho más equilibrada.

	textos	labels
2424	En las zonas rurales de Guatemala, la falta de...	1
2425	En Buenos Aires, la desigualdad económica se m...	1
2426	En Colombia, las comunidades afrodescendientes...	1
2427	En los barrios populares de Lima, la falta de ...	1

Tras unificar el set de datos de la entrega 1, el set de datos de la entrega 2, y los datos generados sintéticamente, se hizo el reentrenamiento del modelo haciendo uso del pipeline del modelo de la entrega 1 (en la sección 2 se explica a detalle la generación del pipeline). Haciendo uso del método fit que traen estos pipelines, fue posible reajustar el modelo a los datos ingresados. Las métricas obtenidas tras el reentrenamiento del modelo fueron las siguientes:

Comparativa de Métricas de Modelos



En este caso podemos observar que todas las métricas mejoran a comparación del primer modelo realizado, adicionalmente, las métricas también presentan valores más deseables que las métricas obtenidas sólo con el segundo set de datos (salvo la Precisión en la clase 1). Teniendo en cuenta lo anterior, podemos concluir que el modelo entrenado con datos generados sintéticamente presenta mejor rendimiento para un entorno de producción, pues todas las métricas mejoran a comparación del modelo anterior (no hay trade-off).

2. Documentación de la Sección 2: Proceso de automatización del proceso de preparación de datos, construcción y persistencia del modelo, y acceso por API

Para la automatización del flujo que se solicitó, se emplearon tres principales archivos, denominados *preprocesamiento.py*, *pipeline_ods.py* y *api.py*.

En primer lugar, el archivo *preprocesamiento.py*, ubicado en la carpeta src, contiene las funciones encargadas de limpiar las opiniones. Estas funciones realizan transformaciones como pasar cada texto a minúsculas, eliminar signos de puntuación, reemplazar números por su equivalente en palabras, tokenizar, lematizar y reducir el ruido que exista mediante la eliminación de stopwords (palabras muy comunes del idioma español, que no ayudan a diferenciar entre tipos de ODS), la eliminación de tildes y el filtrado de tokens vacíos. Para cumplir con la limpieza, se utilizaron librerías como spaCy para hacer lematización y NLTK para quitar las stopwords.

Finalmente, todas las palabras procesadas se vuelven a unir en un único string para que puedan ser utilizadas más adelante por el vectorizador TF-IDF, lo cual se hará en el archivo del pipeline, *pipeline_ods.py*.

Además, *preprocesamiento.py* incluye la función *preprocesar_texto()*, la cual contiene una secuencia donde integra todas las etapas de la limpieza que acabamos de describir, así se ejecutan automáticamente sobre cada uno de los textos antes de pasar a la vectorización. Esto es importante porque de esta manera garantizamos que el ruido se reduzca, exista un formato útil y manejable para la vectorización y aseguramos que el modelo eventualmente reciba entradas homogéneas y útiles para el proceso de clasificación.

Ahora, *pipeline_ods.py*, ubicado en la carpeta src, contiene una clase llamada *PreprocesamientoPersonalizado*, la cual es la encargada de aplicar la función (ubicada en *pipeline_ods.py*) de *preprocesar_texto()* sobre cada una de las opiniones, limpiándolas y dejándolas listas para ir a vectorización. Esta clase se incluye como primer paso en la implementación del pipeline.

Continuamos con el segundo paso del pipeline: la vectorización con TF-IDF, la cual fue creada con los mismos parámetros que en la etapa uno del proyecto, por lo cual se ignoran las palabras que aparecen en menos de dos documentos, pues no aportan información suficiente para que la Regresión Logística las considere determinantes al asignar una ODS. De igual forma, se descartan aquellas que aparecen en más del 90% de los textos, ya que son demasiado frecuentes y no ayudan a diferenciar entre clases. Asimismo, sólo consideramos vectorizar palabras con al menos dos letras, lo que evita incluir números o caracteres aislados que no resultan útiles en el modelo. Con esto, los textos preprocesados quedan representados en una matriz TF-IDF, que tiene pesos, los cuales reflejan qué tan relevante es un término para cada opinión.

El tercer paso es la creación del modelo de Regresión Logística, implementado con los mismos hiperparámetros que en la primera parte del proyecto. Se establece un máximo de 2000 iteraciones, se activa la opción `class_weight="balanced"` para compensar el desbalance en la cantidad de opiniones por ODS, y se utiliza `lbfgs`, que indica que debemos trabajar con un problema de clasificación multiclase, y no binario. Así termina la realización del pipeline.

Finalmente, `pipeline_ods.py` incluye la lógica para entrenar el pipeline completo, utilizando los datos de la etapa uno del proyecto. Tomamos dichos datos, y con la función `crear_conjunto_validacion()`, los cargamos. Una vez cargados, a partir de este conjunto de datos, creamos dos nuevos conjuntos: el conjunto de entrenamiento (80% de los datos originales) y el conjunto de validación (20% de los datos originales). Esto es esencial para más adelante poder evaluar el desempeño del modelo de forma independiente con los datos de validación, que quedan en `val_set.csv`; es una buena práctica, que garantiza que las métricas obtenidas en verdad muestren su capacidad de generalización.

Tomamos el conjunto de datos de entrenamiento y separamos la variable independiente `textos` de la variable objetivo `labels`, para luego ejecutar el pipeline con el flujo del preprocesamiento, vectorización y entrenamiento del modelo de Regresión Logística. Hacemos el entrenamiento y guardamos el pipeline en la carpeta `models` usando la librería `joblib`; este queda con el nombre de `pipeline_ods_model_entrenado.joblib`, por lo cual queda entrenado con los datos base y listo para usarse con nuevas opiniones. Además, guardamos el conjunto de datos originales con los que entrenamos en la carpeta `data`, bajo el nombre de `training_data.csv`, lo que ayuda en caso de querer reentrenar más adelante con una base de datos histórica, donde se irán acumulando los datos usados para entrenar.

Cabe resaltar que la persistencia fue realizada no como parte del pipeline, pues no sería coherente, sino como un proceso posterior al entrenamiento. Así, una vez se han hecho transformaciones, modelo y entrenamiento, es `joblib` la que nos ayuda a que quede persistido. Ahora, la persistencia de los datos usados se hizo en el csv.

De esta manera, el archivo *pipeline_ods.py* deja preparado el modelo que será usado y actualizado mediante la API.

Finalmente, *api.py* se ubica en la carpeta *app*, y es el archivo que se empleó para poder tener acceso al modelo mediante API REST. En este archivo utiliza dos clases: *OpinionesEntrada* y *OpinionEntrenamiento*. Ambas reciben una o más opiniones, y se usarán para predecir y entrenar respectivamente. La primera clase, recibe textos; mientras que, la segunda, recibe textos y su clasificación (labels).

En *api.py* se gestiona la carga del pipeline que persistimos, mediante dos endpoints:

1. **/predecir:** Llegan los textos de *OpinionesEntrada*, así que el pipeline aplica el preprocesamiento, la vectorización TF-IDF y el modelo de Regresión Logística sobre cada una de ellos. Luego, se usa el método *predict_proba()* del modelo para calcular la probabilidad asociada a cada clase y poder regresar un elemento de formato JSON con las predicciones que resultaron para cada opinión, así como los scores (la probabilidad más alta asociada a la clase de cada texto) y la distribución completa de probabilidades por clase.

2. **/entrenar:** Llegan los textos y las etiquetas de *OpinionEntrenamiento*, así que el pipeline se reentrena. Para ello, se usa el histórico de datos de entrenamiento, es decir, aquellas opiniones con etiquetas que ya estaban persistidas en *training_data.csv*, así como las nuevas opiniones y etiquetas. Reentrenamos al modelo apoyados de todo el flujo del pipeline, y persistimos el nuevo conocimiento que la Regresión Logística tiene dentro del pipeline, al emplear la librería *joblib*. Para terminar, evaluamos al modelo que ha sido reentrenado con los datos de validación que anteriormente separamos en *val_set.csv* y entregamos un elemento de formato JSON, que contiene las métricas de desempeño: *precision*, *recall* y *f1-score*.

Resulta importante consultar las métricas para comparar el comportamiento de la Regresión Logística antes y después del reentrenamiento, verificando si el modelo logra mejores resultados, lo cual beneficiaría al Fondo de Población de las Naciones Unidas para obtener clasificaciones más precisas de las opiniones ciudadanas según los Objetivos de Desarrollo Sostenible. Gracias a ello, la organización podría obtener un sistema más confiable para analizar tendencias, ordenar sus prioridades y orientar de manera más efectiva sus planes y programas.

Por último, la API fue desarrollada con FastAPI, un framework que permite conectar modelos de Machine Learning con servicios web mediante endpoints HTTP. Para su ejecución se utilizó el servidor Uvicorn.

3. Descripción de Tres Estrategias de Reentrenamiento

Estrategia 1: Reentrenamiento completo con histórico acumulado

En esta estrategia el modelo de Machine Learning se entrena desde cero cada vez que nuevos datos son ingresados. Una característica particular es que siempre se entrena con el conjunto de datos completo, es decir, tanto con los datos que se usaron anteriormente como base de entrenamiento, como con las nuevas opiniones etiquetadas que han llegado, lo cual se llama base de datos histórica (Zhang et al., 2020).

De esta manera, es posible conservar la información completa, sin pérdidas, así como integrar los nuevos comportamientos y conocimientos.

Una ventaja respecto a esta estrategia es que, al reentrenar sobre datos históricos (antiguos y nuevos), mejora la estabilidad del modelo, y evita perder información importante del pasado, en comparación con otros métodos que suelen ser parciales, y van dejando a un lado estos datos. Todo ello, mientras permite aprender de nuevos casos.

Una desventaja de este enfoque es que suele requerir un tiempo alto de cómputo a medida que la cantidad de datos almacenados en la base de datos histórica crece, lo cual puede resultar costoso.

Esta estrategia fue la que se implementó en el presente proyecto.

Justificación de esta elección: A pesar de que esta estrategia puede resultar más costosa en términos de tiempo de cómputo, la elegimos debido a la estabilidad que brinda. Hacer reentrenamientos con una base de datos completa nos ayuda a conservar el progreso que el modelo ha alcanzado antes en cuanto a la calidad de las clasificaciones. Adicionalmente, esto nos ayuda a que el modelo esté libre de sesgos artificiales ante las opiniones más recientes, porque aún conoce opiniones ciudadanas anteriores, lo cual es demasiado importante para un proyecto con una aplicación social, como es en el caso de la solicitud del Fondo de Población de las Naciones Unidas, pues de otro modo podría favorecerse a alguna de las ODS sin ser fiel a la realidad. Por último, este enfoque abre la posibilidad de que el Fondo de Población de las Naciones Unidas tenga la oportunidad de revisar y analizar tendencias de opinión a lo largo del tiempo, fortaleciendo la transparencia y la guía que les puede proporcionar el sistema.

Estrategia 2: Reentrenamiento con la técnica de la Ventana Deslizante

En esta estrategia, el modelo se reentrena con los datos más recientes y con algunos de los datos antiguos, por lo que los va "desplazando" o "deslizando" poco a poco. Mantiene una ventana o parte de datos históricos activa, por ejemplo, los últimos 5000 datos o los datos del último mes, y actualiza este conjunto a medida que llega nueva información (Devansh, 2024).

Así, el modelo se adapta constantemente a las tendencias sociales en las opiniones ciudadanas, o en general en los contextos de Machine Learning, sin necesidad de conservar toda la base de datos histórica completa.

Una ventaja de esta estrategia es que requiere menor costo de tiempo computacional que reentrenar por completo, así como menos almacenamiento. Igualmente, es capaz de actualizarse frente a cambios recientes, lo cual es útil en casos de noticias, tendencias o redes sociales.

Una desventaja de este método es que puede perder conocimiento importante si los datos antiguos aún son relevantes y la ventana de datos no lo cubre, por lo que definir el tamaño de la ventana requiere muchísimo cuidado. Si es muy pequeña, olvidará rápido datos importantes. Si es muy grande, se volverá costosa.

Estrategia 3: Reentrenamiento incremental o Entrenamiento Continuo

Esta estrategia entrena nuevamente al modelo sin empezar desde cero. Por el contrario, aprende pedacito por pedacito de los datos, en secuencia. Cada uno de esos pedacitos se conoce como batch, así que aprende con un batch de las nuevas opiniones, elige los mejores hiperparámetros y luego recibe el siguiente batch de las nuevas opiniones, teniendo en cuenta el conocimiento anterior (GeeksforGeeks, 2025). Usa únicamente el nuevo conjunto de datos, nunca usa el conjunto de datos antiguo.

Una ventaja respecto a esta estrategia es que es rápida al recibir datos en tiempo real, no suele demorarse, ya que al manejar batches de datos, puede ir procesando poco a poco, sin sobrecargarse y sin reentrenarse con toda la base cada vez. Es útil para sistemas que requieren respuestas rápidas.

Una desventaja es que con esta estrategia no es aceptada por todos los modelos, por ejemplo, la Regresión Logística no se puede incorporar con este método. Igualmente existe el riesgo de olvidar patrones antiguos si los datos no están balanceados o no son representativos, lo cual eventualmente podría conducir a generar un sesgo artificial.

4. Documentación de la Sección 3: Desarrollo de la aplicación y Justificación Descripción del Usuario de la aplicación

Partiremos de la descripción del usuario/rol de la organización que va a utilizar la aplicación:

Siendo la organización, el Fondo de Población de las Naciones Unidas (UNFPA), podemos definir al usuario como un “Analista de Políticas Públicas”, el cual está encargado de revisar y comprender grandes volúmenes de información textual proveniente de distintas fuentes de participación ciudadana para evaluar políticas y entender las distintas problemáticas.

Entre las funciones de este rol destaca la tarea de relacionar los múltiples contenidos textuales con los objetivos de desarrollo sostenible (ODS), la cual puede al ser manual, suele requerir recursos significativos y varios expertos en la tarea, aquí es donde surge la necesidad de esta aplicación.

Para este rol, la importancia de la existencia de la aplicación se encuentra en:

- La automatización de la clasificación de opiniones, permitiendo al analista procesar miles de comentarios en minutos en lugar de dedicarle semanas.
- Identificación de patrones en tiempo real, pues al contar con una aplicación con tan bajo tiempo para dar los resultados, es posible identificar rápidamente qué ODS son más prevalentes en ciertas comunidades, permitiéndoles orientar las políticas públicas de una manera más eficiente.

5. Desarrollo de la Aplicación Web:

Para el desarrollo de la interfaz gráfica de la aplicación, partiendo de la API con los endpoints ya definidos, podemos definir las siguientes funcionalidades claves:

- **Interfaz para Clasificar Opiniones:**

- Un área de texto donde el usuario pueda pegar una o varias opiniones.
- Un botón de "Clasificar".
- Una sección de resultados que muestre cada opinión junto con el ODS predicho y la probabilidad asociada. El *endpoint* `/predecir` de tu API se encargará de esto.

- **Interfaz para Re-entrenar el Modelo (Usuario Experto):**

- Una sección separada, llamada "Mantenimiento del Modelo".
- Un formulario que permita subir un archivo (por ejemplo, un CSV) con nuevas opiniones y sus etiquetas correctas.
- Un botón de "Re-entrenar".
- Al finalizar, la interfaz debería mostrar las nuevas métricas de desempeño (precisión, recall, F1-score) que devuelve el *endpoint* `/entrenar`

6. Resolución de Preguntas

¿Qué recursos informáticos requiere para entrenar, ejecutar, persistir el modelo analítico y desplegar la aplicación?

La aplicación requiere una infraestructura para backend (FastAPI, Uvicorn, Python, modelo de Regresión Logística) y frontend. El modelo se persiste con joblib y los datos históricos en CSV, demandando más recursos con el tiempo. Se recomienda despliegue en la nube con servicios de contenedores/PaaS para el backend, alojamiento estático para el frontend, y almacenamiento de objetos (S3/GCP Storage) para modelo y datos, asegurando escalabilidad y durabilidad.

¿Cómo se integrará la aplicación construida a la organización, estará conectada con algún proceso del negocio o cómo se pondrá a disposición del usuario final?

La aplicación se integrará gradualmente en el UNFPA. Inicialmente, el Analista de Políticas Públicas la usará vía web para clasificar textos de participación y reentrenar el modelo, agilizando un proceso manual y costoso. La herramienta se conecta con la evaluación de políticas y la identificación de problemas territoriales según los ODS. A futuro, la API REST permitirá que otras plataformas del UNFPA consuman la clasificación, conectándose a sistemas de recolección de datos para clasificar opiniones automáticamente, alimentando dashboards y monitoreando necesidades poblacionales en tiempo real.

¿Qué riesgos tiene para el usuario final usar la aplicación construida?

El uso de la aplicación por parte del usuario final en el UNFPA conlleva ciertos riesgos asociados a los modelos de aprendizaje automático. El principal riesgo es la clasificación incorrecta de las opiniones ciudadanas. Un error del modelo podría llevar al analista a sacar conclusiones equivocadas sobre las prioridades de una comunidad, lo que podría desviar la asignación de recursos y el diseño de políticas públicas. Existe el peligro de un exceso de confianza en la herramienta, donde las predicciones se acepten sin un análisis crítico, omitiendo la opinión del analista experto en el tema.

Otro riesgo significativo es el sesgo en los datos con los que el modelo es entrenado y entrenado. Si los datos no son representativos de la diversidad demográfica del contexto, el modelo podría funcionar de manera deficiente para ciertos grupos, invisibilizando sus problemáticas. Finalmente, el modelo tiene una capacidad limitada para entender matices del lenguaje como el sarcasmo o el contexto cultural, lo que podría llevar a interpretaciones erróneas. Es crucial que el usuario final sea consciente de estas limitaciones y utilice la aplicación como una herramienta de apoyo que complemente, y no reemplace, el análisis cualitativo y contextual.

7. Resultados

Se desarrolló un video de 5 minutos (disponible en: <https://www.youtube.com/watch?v=ZGbR7Ztl0ro>) que simula la interacción de un Analista de Políticas Públicas del UNFPA con la aplicación web. El video demuestra el flujo de trabajo completo, mostrando cómo el analista puede priorizar recursos y gestionar el modelo. Concluye reflexionando sobre el impacto potencial en Colombia, agilizando la toma de decisiones y alineando políticas públicas con necesidades reales.

8. Pruebas de Usabilidad

Para validar la aplicación, se realizaron pruebas de facilidad de uso con 10 personas cercanas utilizando el cuestionario estándar de la industria, la Puntuación de

Usabilidad del Sistema (SUS). La plataforma obtuvo una puntuación promedio de 85 sobre 100, una calificación considerada bastante positiva. Este resultado confirma que la aplicación es intuitiva y permite una rápida adopción por parte de los usuarios finales del UNFPA.

9. Roles y tareas realizadas por los integrantes del grupo

Ana María Hernández Lasso: Desarrolló el pipeline completo (preprocesamiento, TF-IDF, Regresión Logística), implementó los endpoints /predecir y /entrenar con persistencia del modelo y actualización del dataset, documentó el código y describió tres estrategias de reentrenamiento. Todo verificado con pruebas de funcionalidad. Dio organización, revisión y mejora al presente documento (8 horas, 33 puntos).

Juan Sebastián Ardila López: Encargado de la evaluación de métricas, construcción de datos sintéticos con prompting, reentrenamiento del modelo con nuevos datos y exportación para producción. Documentó el código y la revisión literaria para la selección de hiperparámetros de prompting, experimentando con temperaturas de gpt-4o-mini. (5 horas, 33 puntos).

William Bayona Vergara: Desarrolló la aplicación (FrontEnd) y sus pantallas según principios de diseño web. Justificó resultados, presentó al usuario la plataforma, produjo el video de muestra y validó la usabilidad con diferentes personas. (8 horas, 33 puntos).

10. Retos enfrentados y formas planteadas para resolverlo

- Se presentaron complicaciones al integrar el pipeline completo dentro de la API, debido a una organización confusa de los archivos en su estructura original. Esto se resolvió ajustando las rutas e importaciones entre carpetas, para mejorar su legibilidad y trabajo.

11. Declaración de uso de Inteligencia Artificial

Ana María Hernández Lasso: Utilización de ChatGPT-5 como apoyo para fragmentos específicos de código (por ejemplo, la inclusión de la probabilidad asociada por clase), así como para la revisión del pipeline y los endpoints. Se empleó para mejorar la redacción.

Juan Sebastián Ardila López: Utilización del API de Open-AI con el modelo gpt-4o-mini para la generación de datos mediante prompting. Uso de Gemini 2.5 PRO para gráficas comparativas entre métricas de los modelos y corrección de errores menores de código.

William Bayona Vergara: Uso de Gemini 2.5 PRO para apoyo de código en la elaboración de componentes de la aplicación, recomendaciones de usabilidad. Mejoras en la redacción del guion del video.

Referencias

Devansh. (2024, 28 de febrero). *Dominando la técnica de las ventanas corredizas*. Medium. Recuperado de <https://medium.com>

GeeksforGeeks. (2025, julio 31). *Incremental Learning with Scikit-learn*. Recuperado de <https://www.geeksforgeeks.org/machine-learning/incremental-learning-with-scikit-learn/>

Holtzman, A., Buys, J., Du, L., Forbes, M., & Choi, Y. (2020). *The Curious Case of Neural Text Degeneration*. In International Conference on Learning Representations (ICLR).
Recuperado de <https://arxiv.org/abs/1904.09751>

Zhang, Y., Liu, C., Zhou, J., & Lin, D. (2020). *How to Retrain Recommender System? A Sequential Meta-Learning Method*. arXiv preprint arXiv:2005.13258. <https://arxiv.org/abs/2005.13258>